

C4 Model do sistema em MVT

Nível 1: Contexto do Sistema

Este nível ilustra a visão macro do sistema, demonstrando como os usuários interagem com a solução de gestão de conformidade para controlar documentos, visitas técnicas e colaboradores.

Usuário Autenticado

Representa qualquer operador do sistema. Não há distinção de níveis de permissão, de modo que todos os usuários autenticados possuem o mesmo nível de visualização e edição dentro da plataforma. O acesso ocorre estritamente mediante usuário e senha.

Sistema de Gestão de Conformidade

É o núcleo da aplicação. O sistema recebe as requisições dos usuários para gerenciar os responsáveis técnicos, empresas, e o controle automático de prazos e vencimentos de documentos.

Ambiente de Armazenamento Local

Representa a infraestrutura física ou de rede do cliente, onde o banco de dados e as rotinas de backup automatizado residem de forma independente.

Nível 2: Containers

Neste nível, o sistema é "aberto" para exibir seus contêineres de execução primários, evidenciando as escolhas tecnológicas focadas em portabilidade e empacotamento.

Aplicação Web Dockerizada

Toda a aplicação em Django (Python) e suas dependências rodam isoladas dentro de um container Docker. Isso garante o funcionamento uniforme do sistema em qualquer computador do cliente. A comunicação com o usuário é feita via navegador padrão.

Variáveis de Ambiente e Configuração

Módulo responsável por injetar as chaves de segurança e apontar dinamicamente o caminho do banco de dados na rede local, sem expor o código-fonte e fora do container Docker.

Banco de Dados Único (SQLite)

O armazenamento de todas as entidades relacionais ocorre em um arquivo de banco de dados SQLite. Este arquivo é mantido em um servidor de arquivos interno indicado pelo cliente e fora do container docker.

Nível 3: Componentes

A arquitetura interna da Aplicação Web Dockerizada segue o padrão MVT (Model, View, Template) nativo do framework Django. Aqui detalhamos as camadas de software responsáveis pelas regras de negócio.

Camada de Template (Interface e Dashboards)

Responsável por renderizar a interface do login e após um login bem-sucedido. O usuário é redirecionado para uma tela inicial que contém opções de cadastros, alertas de conformidade e relatórios estratégicos. Esta camada destaca visualmente documentos ou colaboradores, faltantes ou próximos do vencimento.

Camada de View (Controladores lógicos)

Orquestra as requisições que partem dos Templates. Esta camada processa cálculos automáticos, como a soma do prazo em meses à data de entrega para gerar vencimentos , e o cálculo de validade fixa de um ano para colaboradores.

Camada de Autenticação e Criptografia

Módulo que aplica os algoritmos de hash seguros do Django para as senhas dos usuários. Também é responsável pela criptografia bidirecional aplicada em dados sensíveis previstos pela LGPD, como o CPF de Responsáveis Técnicos e Colaboradores.

Camada de Model (Mapeamento Relacional)

Realiza a comunicação direta com o SQLite e representa todas as tabelas transacionais e de domínio do sistema. Inclui entidades primárias como Usuario, ResponsavelTecnico, Empresa e TipoDocumento , além de estruturas de junção como NecessidadeDocumento, EntregaDocumento e Visita.

Componente Logger Interceptador

Módulo especializado que atua silenciosamente por trás da Camada de Model. Ele escuta todas as operações de mutação de dados para registrar informações vitais de rastreabilidade (“quem”, “quando”, “o que” e o “detalhamento da ação”) na tabela LogAuditoria.

Nível 4: Código

Este nível detalha as classes, atributos e métodos envolvidos nos fluxos de autenticação e de cadastro de entidades no sistema.

O detalhamento abrange o caminho da requisição desde a interface do usuário até a persistência no banco de dados SQLite.

Fluxo de Autenticação de Usuário

O processo inicia quando o usuário acessa o sistema e tenta realizar o login para acessar o painel de controle.

Classe *LoginTemplate* Camada de interface responsável por capturar os dados de acesso digitados pelo usuário e enviar a requisição inicial.

Atributos:

- username
- password

Métodos:

- renderizarTelaLogin()
- enviarFormularioDeAcesso()

Classe *AuthView* Controlador lógico que recebe a requisição do template, orquestra a validação e gerencia o roteamento do usuário.

Atributos:

- httpRequest
- sessaoAtiva

Métodos:

- processarPostLogin()
- redirecionarParaHome()

Classe *AuthService* Módulo de segurança e criptografia que aplica os algoritmos nativos do Django para proteger e validar senhas e dados sensíveis como CPFs.

Atributos:

- algoritmoHash
- dadosCriptografados

Métodos:

- validarCredenciais()
- compararHashSenha()
- converterCPF()

Classe *UserModel* Mapeamento relacional que acessa a tabela de usuários no banco de dados para confirmar a existência do registro.

Atributos:

- idUsuario
- username
- passwordHash

Métodos:

- buscarPorUsername()
- verificarStatusAtivo()

Após a validação bem-sucedida passando pelo *AuthService* e *UserModel*, a *AuthView* aprova o acesso e retorna a resposta redirecionando o usuário para a tela inicial.

Classe *HomeTemplate* Interface principal exibida após o login, contendo os atalhos de cadastros e os dashboards de alertas e relatórios.

Atributos:

- listaAlertasConformidade
- listaRelatoriosDisponiveis

Métodos:

- renderizarDashboardInicial()
- navegarParaOpcoes()

Fluxo de Cadastro de Empresa com Auditoria

A partir da tela inicial, o usuário acessa o formulário para registrar uma nova empresa, acionando o fluxo de persistência que inclui a interceptação do logger.

Classe *EmpresaTemplate* Interface inicial com listagem de todos os cadastros de empresas dessa tela o usuário pode editar um registro, apagar um registro, cadastrar um registro novo, ver as dependências de uma empresa ou ver o histórico de visitas de uma empresa.

Atributos:

- ListaDeEmpresas

Métodos:

- buscarEmpresas()
- editarRegistro()
- apagarRegistro()
- cadastrarNovoRegistro()
- verDependencias()
- verHistoricoVisitas()

Classe *EmpresaCadastroTemplate* Interface de formulário para preenchimento dos dados cadastrais da empresa e definição do seu responsável técnico.

Atributos:

- nomeEmpresa
- enderecoEmpresa
- cnpjEmpresa
- fkResponsavelTecnico
- listaResponsaveisTecnicos

Métodos:

- renderizarFormularioCadastro()
- listarResponsaveisTecnicos()
- enviarDadosParaView()

Classe *EmpresaView* Recebe os dados preenchidos no formulário, aplica as validações de regras de negócio e solicita a gravação.

Atributos:

- httpRequest
- dadosFormularioEmpresa

Métodos:

- processarPostCadastro()
- validarRegrasNegocio()
- retornarSucessoTelaInicial()

Classe *EmpresaModel* Representação da tabela de empresas no banco de dados SQLite, responsável por executar as instruções SQL.

Atributos:

- idEmpresa
- nome
- endereco
- cnpj
- fkRespTecnico

Métodos:

- salvarNovoRegistro()
- alterarRegistroExistente()
- deletarRegistro()

Classe *LoggerInterceptor* Classe que atua nos bastidores da camada Model para registrar qualquer mutação de dados garantindo a rastreabilidade.

Atributos:

- estadoOriginalDado
- estadoNovoDado
- idUsuarioRequisicao
- tipoOperacaoExecutada

Métodos:

- interceptarRequisicaoModel()
- capturarEstadoOriginal()
- montarObjetoDeLog()
- dispararGravacaoAssincrona()

Classe *LogAuditoriaModel* Entidade responsável por receber o objeto do interceptor e persistir o histórico de ações na tabela de log.

Atributos:

- idLog
- fkUsuario
- codigoRegistroAfetado
- tipoAcao
- dataHoraOperacao
- descricaoDetalhada

Métodos:

- inserirRegistroAuditoria()

Ao finalizar a persistência pela *EmpresaModel* e o disparo assíncrono pelo *LoggerInterceptor*, a *EmpresaView* devolve uma mensagem de sucesso para a interface, retornando o usuário à tela inicial do *EmpresaTemplate*.